3. HotStuff 합의 알고리즘 구현

3.1 합의 인터페이스 설계

3.1.1 Rules 인터페이스

```
type Rules interface {
// 블록에 투표할지 결정
VoteRule(proposal hotstuff.ProposeMsg) bool
// 블록의 조상 중 커밋 가능한 블록 반환
CommitRule(*hotstuff.Block) *hotstuff.Block
// 커밋에 필요한 체인 길이
ChainLength() int
}
```

3.1.2 ProposeRuler 인터페이스 (선택적)

```
type ProposeRuler interface {
// 새로운 제안 생성
ProposeRule(cert hotstuff.SyncInfo, cmd hotstuff.Command) (proposal hotstuff.Pro
}
```

설계 철학:

- 핵심 합의 로직만 구현하면 됨
- 네트워크 통신, 암호화 등은 기본 구현에서 처리
- 새로운 합의 알고리즘 구현이 용이

3.2 기본 합의 구현 (ConsensusBase)

3.2.1 구조체 정의

```
type consensusBase struct {
impl Rules // 실제 합의 규칙 구현
// 의존성 모듈들
acceptor modules.Acceptor
blockChain modules.BlockChain
commandQueue modules.CommandQueue
configuration modules.Configuration
crypto modules.Crypto
eventLoop *eventloop.EventLoop
executor modules.ExecutorExt
synchronizer modules.Synchronizer
// 상태 변수들
lastVote hotstuff.View // 마지막 투표 뷰
bExec *hotstuff.Block // 마지막 실행된 블록
mut sync.Mutex // 동시성 제어
```

```
}
```

3.2.2 제안 생성 과정

```
func (cs *consensusBase) Propose(cert hotstuff.SyncInfo) {
cs.logger.Debug("Propose")
// 1. 이전 제안 성공 알림
if qc, ok := cert.QC(); ok {
if qcBlock, ok := cs.blockChain.Get(qc.BlockHash()); ok {
cs.acceptor.Proposed(qcBlock.Command())
}
}
// 2. 명령 큐에서 명령 가져오기
ctx, cancel := synchronizer.TimeoutContext(cs.eventLoop.Context(), cs.eventLoop)
defer cancel()
cmd, ok := cs.commandQueue.Get(ctx)
if !ok {
cs.logger.Debug("Propose: No command")
return
}
// 3. 제안 생성
var proposal hotstuff.ProposeMsg
if proposer, ok := cs.impl.(ProposeRuler); ok {
```

```
// 사용자 정의 제안 규칙 사용
proposal, ok = proposer.ProposeRule(cert, cmd)
if !ok {
return
}
} else {
// 기본 제안 생성
proposal = cs.createDefaultProposal(cert, cmd)
}
// 4. 블록 저장 및 전파
cs.blockChain.Store(proposal.Block)
cs.configuration.Propose(proposal)
// 5. 자기 투표
cs.OnPropose(proposal)
}
```

3.2.3 제안 처리 과정

```
func (cs *consensusBase) OnPropose(proposal hotstuff.ProposeMsg) {
block := proposal.Block
cs.logger.Debugf("OnPropose: %v", block)
// 1. AggregateQC 검증 (Fast-HotStuff)
if  cs.opts.ShouldUseAggQC() && proposal.AggregateQC != nil {
```

```
if !cs.verifyAggregateQC(proposal.AggregateQC, block) {
return
}
}
// 2. QC 검증
if !cs.crypto.VerifyQuorumCert(block.QuorumCert()) {
cs.logger.Info("OnPropose: invalid QC")
return
}
// 3. 리더 검증
expectedLeader := cs.leaderRotation.GetLeader(block.View())
if  proposal.ID != expectedLeader {
cs.logger.Info("OnPropose: block was not proposed by the expected leader")
return
}
// 4. 투표 규칙 적용
if !cs.impl.VoteRule(proposal) {
cs.logger.Info("OnPropose: Block not voted for")
return
}
// 5. 명령 수락 확인
if !cs.acceptor.Accept(block.Command()) {
cs.logger.Info("OnPropose: command not accepted")
```

```
return
}
// 6. 블록 저장
cs.blockChain.Store(block)
// 7. 커밋 규칙 적용
if  commitBlock := cs.impl.CommitRule(block);  commitBlock != nil {
cs.commit(commitBlock)
}
// 8. 뷰 진행
cs.synchronizer.AdvanceView(hotstuff.NewSyncInfo().WithQC(block.QuorumCert()))
// 9. 투표 생성 및 전송
cs.createAndSendVote(block)
}
```

3.3 Chained HotStuff 구현

3.3.1 구조체와 상태

```
type ChainedHotStuff struct {
blockChain modules.BlockChain
logger logging.Logger
bLock *hotstuff.Block // 현재 잠긴 블록
```

```
}
```

3.3.2 투표 규칙 (VoteRule)

```
func (hs *ChainedHotStuff) VoteRule(proposal hotstuff.ProposeMsg) bool {
block := proposal.Block
// QC가 참조하는 블록 가져오기
qcBlock, haveQCBlock := hs.blockChain.Get(block.QuorumCert().BlockHash())
safe := false
// 활성 조건: QC 블록의 뷰가 잠긴 블록보다 높음
if haveQCBlock && qcBlock.View() > hs.bLock.View() {
safe = true
} else {
hs.logger.Debug("OnPropose: liveness condition failed")
// 안전성 조건: 현재 블록이 잠긴 블록을 확장
if hs.blockChain.Extends(block, hs.bLock) {
safe = true
} else {
hs.logger.Debug("OnPropose: safety condition failed")
}
}
return safe
}
```

3.3.3 커밋 규칙 (CommitRule)

```
func (hs *ChainedHotStuff) CommitRule(block *hotstuff.Block) *hotstuff.Block {
// 3단계 체인 확인: block -> block1 -> block2 -> block3
// 1단계: block의 QC가 참조하는 블록 (PRE-COMMIT)
block1, ok := hs.qcRef(block.QuorumCert())
if !ok {
return  nil
}
hs.logger.Debug("PRE_COMMIT: ", block1)
// 2단계: block1의 QC가 참조하는 블록 (COMMIT)
block2, ok := hs.qcRef(block1.QuorumCert())
if !ok {
return  nil
}
// 잠금 업데이트
if  block2.View() > hs.bLock.View() {
hs.logger.Debug("COMMIT: ", block2)
hs.bLock = block2
}
// 3단계: block2의 QC가 참조하는 블록 (DECIDE)
block3, ok := hs.qcRef(block2.QuorumCert())
if !ok {
```

```
return nil
}
// 연속된 체인 확인
if block1.Parent() == block2.Hash() &&
block2.Parent() == block3.Hash() {
hs.logger.Debug("DECIDE: ", block3)
return block3 // 커밋할 블록
}
return nil
}
```

3.3.4 3단계 파이프라인 과정

```
뷰 1: [Genesis] <- QC0
뷰 2: [Block1] <- QC1 (Genesis에 대한 QC)
뷰 3: [Block2] <- QC2 (Block1에 대한 QC) // Block1 PRE-COMMIT
뷰 4: [Block3] <- QC3 (Block2에 대한 QC) // Block2 PRE-COMMIT, Block1 COMMIT
뷰 5: [Block4] <- QC4 (Block3에 대한 QC) // Block3 PRE-COMMIT, Block2 COMMIT, Blo
```

3.4 Fast HotStuff 구현

3.4.1 구조체 정의

```go
type FastHotStuff struct {
blockChain modules.BlockChain
logger logging.Logger
synchronizer modules.Synchronizer
}
```

3.4.2 초기화 (AggregateQC 활성화)

```go
func (fhs *FastHotStuff) InitModule(mods *modules.Core) {
var  opts *modules.Options
mods.Get(&opts, &fhs.blockChain, &fhs.logger, &fhs.synchronizer)
// AggregateQC 사용 설정
opts.SetShouldUseAggQC()
}
```

3.4.3 투표 규칙 (VoteRule)

```go
func (fhs *FastHotStuff) VoteRule(proposal hotstuff.ProposeMsg) bool {
// AggregateQC가 있는 경우
if  proposal.AggregateQC != nil {
hqcBlock, ok := fhs.blockChain.Get(proposal.Block.QuorumCert().BlockHash())
return  ok && fhs.blockChain.Extends(proposal.Block, hqcBlock)
}
```

```go
// 일반적인 경우: 뷰 순서 확인
return proposal.Block.View() >= fhs.synchronizer.View() &&
proposal.Block.View() == proposal.Block.QuorumCert().View()+1
}
```

3.4.4 커밋 규칙 (2단계 체인)

```go
func (fhs *FastHotStuff) CommitRule(block *hotstuff.Block) *hotstuff.Block {
// 2단계 체인: block -> parent -> grandparent
parent, ok := fhs.qcRef(block.QuorumCert())
if !ok {
return nil
}
fhs.logger.Debug("PRECOMMIT: ", parent)
grandparent, ok := fhs.qcRef(parent.QuorumCert())
if !ok {
return nil
}
// 연속된 뷰와 부모-자식 관계 확인
if block.Parent() == parent.Hash() &&
block.View() == parent.View()+1 &&
parent.Parent() == grandparent.Hash() &&
parent.View() == grandparent.View()+1 {
```

```
fhs.logger.Debug("COMMIT: ", grandparent)
return grandparent
}
return nil
}
```

3.5 Simple HotStuff 구현

3.5.1 구조체와 상태

```
type SimpleHotStuff struct {
blockChain modules.BlockChain
logger logging.Logger
synchronizer modules.Synchronizer
locked *hotstuff.Block // 잠긴 블록
}
```

3.5.2 투표 규칙 (단순화된 버전)

```
func (hs *SimpleHotStuff) VoteRule(proposal hotstuff.ProposeMsg) bool {
block := proposal.Block
// 규칙 1: 증가하는 뷰에서만 투표
if block.View() < hs.synchronizer.View() {
```

```
hs.logger.Info("VoteRule: block view too low")
return false
}
// 부모 블록 가져오기
parent, ok := hs.blockChain.Get(block.QuorumCert().BlockHash())
if !ok {
hs.logger.Info("VoteRule: missing parent block")
return false
}
// 규칙 2: 부모의 뷰가 잠긴 블록의 뷰보다 크거나 같아야 함
if  parent.View() < hs.locked.View() {
hs.logger.Info("OnPropose: parent too old")
return false
}
return true
}
```

3.5.3 커밋 규칙 (단순화된 3단계)

```
func (hs *SimpleHotStuff) CommitRule(block *hotstuff.Block) *hotstuff.Block {
// 증조부모 블록 커밋 고려
// parent (block의 QC가 참조)
p, ok := hs.blockChain.Get(block.QuorumCert().BlockHash())
```

```go
if !ok {
return nil
}
// grandparent (parent의 QC가 참조)
gp, ok := hs.blockChain.Get(p.QuorumCert().BlockHash())
if ok && gp.View() > hs.locked.View() {
hs.locked = gp
hs.logger.Debug("Locked: ", gp)
} else if !ok {
return nil
}
// great-grandparent (grandparent의 QC가 참조)
ggp, ok := hs.blockChain.Get(gp.QuorumCert().BlockHash())
// 커밋 조건: ggp.view + 2 == p.view (2뷰 차이)
if ok && ggp.View()+2 == p.View() {
return ggp
}
return nil
}
```

3.6 커밋 처리 과정

3.6.1 커밋 실행

```go
func (cs *consensusBase) commit(block *hotstuff.Block) {
cs.mut.Lock()
err := cs.commitInner(block)
cs.mut.Unlock()
if err != nil {
cs.logger.Warnf("failed to commit: %v", err)
return
}
// 블록체인 정리 및 포크 처리
forkedBlocks := cs.blockChain.PruneToHeight(block.View())
for _, forkedBlock := range forkedBlocks {
cs.forkHandler.Fork(forkedBlock)
}
}
```

3.6.2 재귀적 커밋 (commitInner)

```go
func (cs *consensusBase) commitInner(block *hotstuff.Block) error {
// 이미 실행된 블록은 건너뛰기
if cs.bExec.View() >= block.View() {
return nil
}
// 부모 블록 먼저 커밋 (재귀)
```

```
if parent, ok := cs.blockChain.Get(block.Parent()); ok {
if err := cs.commitInner(parent); err != nil {
return err
}
} else {
return fmt.Errorf("failed to locate block: %s", block.Parent())
}
// 지연 시간 측정
cs.eventLoop.AddEvent(hotstuff.ConsensusLatencyEvent{
Latency: time.Since(block.Timestamp())
})
// 블록 실행
cs.logger.Debug("EXEC: ", block)
cs.executor.Exec(block)
cs.bExec = block
return nil
}
```

3.7 투표 생성 및 전송

3.7.1 투표 생성

```
func (cs *consensusBase) createAndSendVote(block *hotstuff.Block) {
```

```
// 중복 투표 방지
if  block.View() <= cs.lastVote {
cs.logger.Info("OnPropose: block view too old")
return
}
// 부분 인증서 생성
pc, err := cs.crypto.CreatePartialCert(block)
if  err != nil {
cs.logger.Error("OnPropose: failed to sign block: ", err)
return
}
cs.lastVote = block.View()
// 투표 전송
cs.sendVote(pc)
}
```

3.7.2 투표 전송 로직

```
func (cs *consensusBase) sendVote(pc hotstuff.PartialCert) {
// Kauri 트리 구조 사용 시
if  cs.kauri != nil {
cs.kauri.Begin(pc, proposal)
return
```

```
}
// 다음 뷰의 리더에게 전송
leaderID := cs.leaderRotation.GetLeader(cs.lastVote + 1)
// 자신이 리더인 경우 이벤트로 처리
if  leaderID == cs.opts.ID() {
cs.eventLoop.AddEvent(hotstuff.VoteMsg{
ID: cs.opts.ID(),
PartialCert: pc
})
return
}
// 다른 복제본에게 전송
if  leader, ok := cs.configuration.Replica(leaderID);  ok {
leader.Vote(pc)
} else {
cs.logger.Warnf("Replica with ID %d was not found!", leaderID)
}
}
```

3.8 안전성과 활성 보장

3.8.1 안전성 메커니즘

- **QC 검증**: 모든 QC의 서명과 쿼럼 크기 확인
- **체인 확장**: 새 블록이 기존 체인을 올바르게 확장하는지 확인
- **리더 검증**: 올바른 리더가 제안했는지 확인
- **뷰 순서**: 뷰 번호의 단조 증가 보장

3.8.2 활성 메커니즘

- **타임아웃**: 진행이 멈추면 다음 뷰로 이동
- **뷰 동기화**: 모든 복제본이 같은 뷰에서 작업
- **리더 교체**: 라운드 로빈 또는 다른 방식으로 리더 교체
- **명령 큐**: 지속적인 명령 공급으로 진행 보장

이러한 구현을 통해 HotStuff는 비잔틴 환경에서도 안전하고 활성을 보장하는 합의를 달성합니다.